SeatWise — Phase 5 Summary

Real Payment — Stripe Checkout, Webhook, Ticket Issuance

What This Phase Proves

The mock payment flow (sessionStorage, a fake /payment page, a Math.random() booking code) is now replaced end to end with real Stripe Checkout, a signature-verified webhook, and actual order/ticket rows in Postgres. A held seat only becomes a permanent, signed ticket after Stripe confirms real payment — closing the exact gap identified earlier in the project, where a confirmed booking screen did not correspond to any real database state.

What Was Built

Item	Purpose
src/lib/stripe.ts	Stripe SDK client, initialized from STRIPE_SECRET_KEY.
createCheckout (src/lib/db/seats.ts)	Server Action: verifies the buyer's active holds for the event, recomputes the price server-sid
src/app/api/webhooks/stripe/route.ts	Verifies the Stripe webhook signature, handles checkout.session.completed, marks the orde
getOrderStatus (src/lib/db/seats.ts)	Lets the frontend poll for real payment confirmation after the Stripe redirect, rather than trust
TicketDrawer.tsx	Old mock-payment sessionStorage flow removed entirely. confirmBooking now calls createC

Problems Faced and Fixed

1. Credential handling. A live Stripe secret key and a Neon database password were pasted in plaintext into chat on two separate occasions during this phase. Both should be rotated; this is a process risk independent of the code itself and is called out explicitly here rather than left implicit.

2. Hold expiry timestamp bug, caught from the data itself. seat_holds rows showed expires_at exactly 5 hours 30 minutes later than intended (IST offset) instead of the intended 10-minute window. Root cause: expiresAt was computed in JavaScript (new Date(Date.now() + ...)) and passed as a parameter, while created_at was set by Postgres's own now(). Two different clocks computing two halves of the same guarantee will drift. Fixed by removing the JS-side computation entirely and letting Postgres compute both timestamps: now() + interval '10 minutes' directly in the SQL.

3. Webhook route returning 404 — root cause was a misplaced file, not a code bug. The webhook route was, at various points, either missing, containing the wrong file's content (the Better Auth handler pasted in by mistake), or physically nested inside src/app/api/auth/[...all]/ instead of sitting at its own path, src/app/api/webhooks/stripe/. Next.js correctly returned 404 for a route that didn't exist at the requested path; the fix was locating and recreating the file at the correct location, not a logic change.

4. PowerShell silently deleted the wrong thing. While cleaning up the misplaced webhook file, a Remove-Item command targeting a path containing [...all] used PowerShell's default -Path parameter, which interprets square brackets as wildcard pattern syntax rather than literal characters. This caused the command to silently affect something other than intended, deleting the actual Better Auth route handler (route.ts) with no error message — breaking login/signup app-wide. Diagnosed with Test-Path, confirmed the file was genuinely gone (not a display or

caching issue), and fixed by recreating the handler and switching all subsequent path operations on that folder to -LiteralPath, which treats brackets as literal text. This is a real, easy-to-repeat PowerShell trap for any Next.js project using bracketed dynamic route folders — worth remembering for any future work in this codebase.

5. A Turbopack crash was a symptom of #3, not a separate bug. With a route file nested inside a catch-all segment ([...all]), Turbopack could not build a valid route tree at all and crashed the entire dev server with a panic error, taking the whole site down — not just the webhook endpoint. Resolved once the misplaced file was correctly removed and the legitimate route restored in its own location.

Verification Performed

Check	Result
Bare POST to /api/webhooks/stripe with no signature	400 (correct — signature required), confirming the route exists and is reac
Real Stripe test-mode Checkout session, correct event name, confirmed price	Completed successfully — price matched server-computed total, not any client
stripe listen forwarding of checkout.session.completed	200 response from the local webhook handler
orders table after payment	status = paid, correct stripe_session_id
tickets table after payment	One row per paid seat, correct event_id/seat_id/order_id, status = valid, re
Frontend polling after Stripe redirect	Printing animation shown, then the confirmed-ticket screen rendered with t

Known Gap Carried Forward (Explicitly, Not Silently)

The webhook's cleanup step currently deletes all seat_holds rows for the event (DELETE FROM seat_holds WHERE event_id = eventId), not just the specific seats belonging to the paid order. For an event with only one active buyer at a time this is harmless, but for a real multi-buyer event, a single successful payment would incorrectly release every other buyer's active holds on that event too. This needs to be scoped to AND seat_id IN (...) using the specific seat list from the order before this is safe under real concurrent multi-buyer load — flagged here so it isn't mistaken for already handled.